

SOFTWARE ENGINEERING (TH3)

1.0 Introduction to Software Engineering

- 1.1 Program vs. Software product
- 1.2 Emergence of Software Engineering.
- 1.3 Computer Systems Engineering
- 1.4 Software Life Cycle Models
 - 1.4.1 Classical Water fall model
 - 1.4.2 Iterative Water fall model
 - 1.4.3 Prototyping model
 - 1.4.4 Evolutionary model
 - 1.4.5 Spiral model

1.1: Program vs. Software Product (2 mark (2020,2022,2024))

1. Core Definitions

- **Program:** A set of instructions written in a programming language (like C, Java, Python) to solve a specific, usually small, problem. It is often written by a single person for personal use.
- **Software Product:** A collection of programs, documentation, and operating procedures developed for a specific set of users or the market. It is robust, tested, and documented.

2. The Equation

Software = Program + Documentation + Operating Procedures

Feature	Program	Software Product
Size	Small (hundreds of lines of code).	Large (thousands to millions of lines of code).
Developer	Developed by a single individual (author).	Developed by a team of developers.
User	usually the author is the only user.	Large, distinct external user base.
Documentation	Lack of proper documentation.	Well-documented (includes SRS, Design docs, User Manuals).
Interface	User Interface (UI) is often missing or crude.	Well-designed, user-friendly Interface (UI).

1.2 Emergence of Software Engineering. 0 times
1.3 Computer Systems Engineering 0 times

1.4 Software Life Cycle Models

1.4.1 Classical Water fall model

Definition The Classical Waterfall Model is the oldest and most theoretical software development life cycle (SDLC) model. It is a **linear-sequential** model, meaning the development moves in only one direction (downwards), like a waterfall.

Key Rule: You must complete one phase entirely before starting the next. You cannot go back to a previous phase.



Phase 1: Feasibility Study

- **Goal:** Determine if the project is practically possible to build within the budget and time.
- **Activities:**
 - Analyze the problem roughly.
 - Check **Technical Feasibility** (Do we have the technology?).
 - Check **Financial Feasibility** (Is it within budget?).
- **Output:** Feasibility Report.

Phase 2: Requirement Analysis & Specification (SRS)

- **Goal:** Understand *exactly* what the customer wants.
- **Activities:**

- Gather requirements from the client.
 - Analyze requirements for inconsistencies.
 - Document everything in the **SRS (Software Requirement Specification)** document.
- **Output:** SRS Document (This serves as a contract between developer and client).

Phase 3: Design

- **Goal:** Plan *how* to build the system based on the SRS.
- **Activities:**
 - **High-Level Design (System Design):** Defining the architecture, modules, and data flow.
 - **Low-Level Design (Detailed Design):** Designing internal logic of each module (algorithms, data structures).
- **Output:** Design Documents.

Phase 4: Coding & Unit Testing

- **Goal:** Translate the design into actual code.
- **Activities:**
 - Programmers write code in languages like C, Java, or Python.
 - **Unit Testing:** Each module is tested individually by the developer to check if it works correctly in isolation.
- **Output:** Source Code.

Phase 5: Integration & System Testing

- **Goal:** Combine modules and test the whole system.
- **Activities:**
 - **Integration:** Joining separate modules (e.g., combining the "Login" module with the "Database" module).
 - **System Testing:** Checking if the entire integrated system meets the requirements specified in the SRS.
- **Output:** A working Software Product

Phase 6: Maintenance

- **Goal:** Keep the software running after delivery.
- **Activities:**
 - **Corrective Maintenance:** Fixing bugs found by the user after delivery.
 - **Perfective Maintenance:** Adding new features or improving performance.
 - **Adaptive Maintenance:** Porting the software to a new environment (e.g., new OS).
- **Note:** This is the longest phase of the lifecycle.

Advantages	Disadvantages
Simple & Easy: Easy to understand and manage.	No Feedback Path: Real projects rarely follow a straight line. If you find an error in Testing, you can't easily go back to Design.

Advantages	Disadvantages
Discipline: Phases are clearly defined with start and end points.	Rigid: It assumes requirements are fixed. If the customer changes their mind later, the model fails.
Documentation: Enforces documentation at every stage.	Late Delivery: The customer sees the working product only at the very end.
Clear Milestones: Good for project management tracking.	Blocking States: Developers may sit idle waiting for designers to finish.

4. When to use this model?

Use the Classical Waterfall Model **only** when:

- Requirements are **very well known** and fixed (e.g., migrating an old system to a new language).
- The project is **short**.
- The technology is well understood.

1.4.2 Iterative Water fall model

1.4.2: Iterative Waterfall Model

1. The Concept (Waterfall + Feedback)

The Iterative Waterfall Model is essentially the Classical Waterfall Model with feedback loops added.

- In the Classical model, you can never go back.
- In the Iterative model, if an error is detected in a later phase (e.g., Testing), you can return to a previous phase (e.g., Design or Requirements) to fix it¹.

2. How it Works

- Development still proceeds sequentially (Requirements $\rightarrow$ Design $\rightarrow$ Code).
- However, there are **backward arrows** (Feedback Paths) from every phase to its preceding phases.
- **Example:** If a programmer finds a logic error during **Coding**, they can go back to the **Design** phase, correct the design document, and then update the code.

Feature	Classical Waterfall	Iterative Waterfall
Feedback	No feedback paths (Rigid).	Has feedback paths (Flexible).

Feature	Classical Waterfall	Iterative Waterfall
Error Cost	Very high (must restart).	Lower (can go back and fix).
Usage	Theoretical/Ideal projects.	Practical/Real-world projects.

"Phase Containment of Error"

(Note: This term appeared in 2021 and 2024 exams as a 2-mark question. It is closely related to this model.)

- **Definition:** The principle of detecting and correcting an error in the **same phase** where it was created, rather than letting it (leak) propagate to the next phase. The Iterative model attempts to achieve this but allows for correction if the error slips through.

1.4.3: Prototyping Model

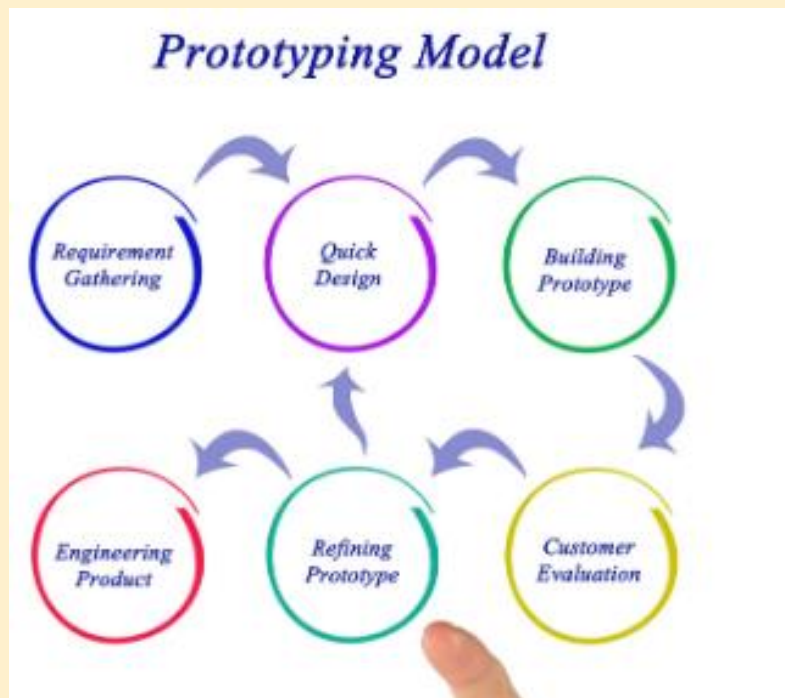
1. What is a Prototype?

A **prototype** is a working model or a "toy implementation" of the software system. It is built quickly to show the customer what the final product will look like and how it will function. It is not the final software—it lacks full functionality, security, and robustness.

2. How the Model Works

This model is used when the customer cannot clearly define their requirements at the beginning.

1. **Requirement Gathering:** Developer and customer define overall objectives.
2. **Quick Design:** A quick design focuses only on the visible parts (User Interface).
3. **Build Prototype:** A working model is built.
4. **Customer Evaluation:** The customer uses the prototype and gives feedback.
5. **Refinement:** The developer fixes errors and adds missing features based on feedback.
6. **Final Product:** Once the customer is satisfied with the prototype, the actual high-quality software is built (often using the Waterfall approach).



Advantages (Memorize these for 2-mark questions)

- **Better Requirement Clarity:** Users can "see" the system, which helps them understand and articulate their needs better.
- **Early Error Detection:** Misunderstandings between the client and developer are caught immediately during the evaluation phase.
- **User Satisfaction:** The customer feels involved in the development process and is less likely to reject the final product.
- **Reduced Risk:** Lowers the risk of building a product that nobody wants.

Disadvantages

- **Cost & Time:** Building a prototype requires extra money and effort upfront.
- **"Throwaway" Code:** Often, the prototype is discarded to build the final quality software, which developers may see as a waste.
- **False Expectations:** Customers may mistake the prototype for the final product. When the developer says, "I need 6 months to build the real one," the customer gets confused (this is often called the "99% complete syndrome").

When to use the Prototyping Model?

(Asked as a 10-mark question in 2022 & 2024) You should use this model **under these circumstances:**

1. **Unclear Requirements:** When the customer knows *what* they want generally but cannot give detailed input/output definitions.
2. **Complex User Interface:** When the interaction (UI/UX) is the most critical part of the system.
3. **New Technology:** When the developers are using a new technology and want to test its capabilities first.

1.4.4: Evolutionary Model

1. The Core Concept

The Evolutionary Model is based on the idea that software should evolve over time, just like a living organism. Instead of building the entire system at once (like in the Waterfall model), you build it in increments or versions.

- **Key Difference from Prototyping:** In the Prototyping model, you build a "dummy" and throw it away. In the Evolutionary model, you build a core version, give it to the customer, and then **keep improving it** until it becomes the final product.

2. How it Works

1. **Initial Version:** Developers build a core version of the system with limited functionality (the most important features only).
2. **Delivery:** This version is released to the user.
3. **Feedback:** The user works with the system and provides feedback or asks for new features.
4. **Evolution:** The developers take this feedback, add new features, fix bugs, and release **Version 2**.
5. **Repeat:** This cycle continues (Version 1 $\rightarrow$ Version 2 $\rightarrow$ Version 3...) until the full system is complete.

3. Visualizing the Process

Imagine building a house:

- **Waterfall:** You wait until every room, window, and paint job is finished before you move in.
- **Evolutionary:** You build just the kitchen and bedroom first. You move in. Next month, you add the living room. The month after, you add the garage. The house "evolves" while you are using it.

4. When to use it?

- **Large Projects:** When the system is too big to be done in one go.
- **Unclear Long-term Requirements:** When the customer knows what they need *now*, but isn't sure what they will need *next year*.
- **Market Pressure:** When a company needs to get a product to market quickly (even with fewer features) to beat competitors.

5. Advantages & Disadvantages

- **Advantages:**
 - Users get to use the software very early.
 - Errors are easy to spot in smaller versions.
 - Training users is easier because they learn the system piece by piece.
- **Disadvantages:**

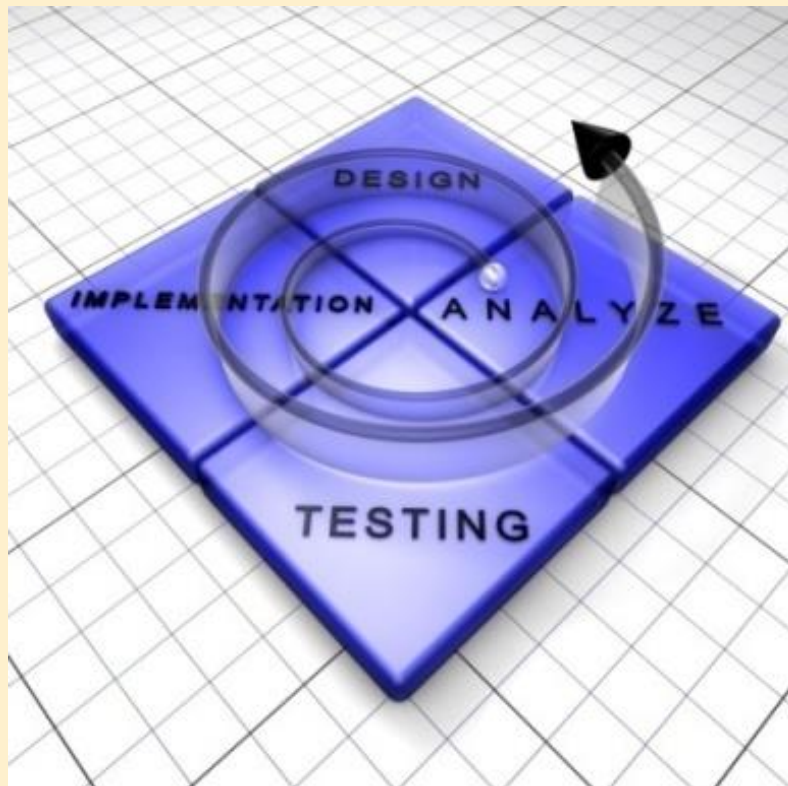
- **"Spaghetti Code"**: Constantly adding new features to old code can make the software messy and unstructured if not managed well.
- **Hard to Budget**: It is difficult to predict exactly how long the project will take since the "evolution" can go on indefinitely.

1.4.5: The Spiral Model

1. Definition

The Spiral Model, proposed by Barry Boehm, is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the Waterfall model.

- **Key Feature**: Its primary focus is on **Risk Analysis**. It is the only model that explicitly includes a risk assessment step in every loop.



Structure: The 4 Quadrants (Phases)

The model is represented as a spiral, where each loop represents a phase of the software process. The spiral is divided into four quadrants:

1. **Planning (Objective Setting):**

- **Goal:** Determine objectives, alternatives, and constraints for this specific loop.
- **Activity:** The team decides what needs to be achieved in this iteration (e.g., "Fix the user interface" or "Build the database module").

2. **Risk Analysis (The Unique Phase):**

- **Goal:** Identify and resolve risks.
- **Activity:** Technical and management risks are assessed. **Prototypes** are often built here to verify if the requirements are feasible. If risks are too high, the project might be terminated here.

3. **Engineering (Development & Testing):**

- **Goal:** Build the product.
- **Activity:** The actual development happens here. Depending on the risk, this might look like the Waterfall model (Design $\rightarrow$ Code $\rightarrow$ Test).

4. **Evaluation (Customer Review):**

- **Goal:** Get feedback.
- **Activity:** The customer evaluates the work produced in the Engineering phase. Based on this feedback, the planning for the *next* loop begins.

3. Why is the Spiral Model called a "Meta-Model"?

(Memorize this section for the 10-mark question)

The Spiral Model is called a **Meta-Model** because it is a "model of models." It does not force you to use a single fixed process; instead, it subsumes (includes) other models based on the situation.

- **It uses Prototyping:** At the start of every loop, specifically during the "Risk Analysis" phase, it uses the **Prototyping Model** to reduce uncertainty.
- **It uses Waterfall:** Once risks are resolved, the "Engineering" phase often follows the standard **Waterfall** steps (Design, Code, Test) to build the feature.
- **It adapts:**
 - If the risk is low and requirements are fixed, the spiral can behave like the **Waterfall Model**.
 - If the risk is high and requirements are unclear, it behaves like the **Prototyping Model**.

Conclusion: Because it provides a framework that can contain and manage other models within itself, it is the "Meta-Model."

4. Advantages

- **Risk Handling:** It is the best model for high-risk projects because risks are analyzed at every step.
- **Flexibility:** It accommodates changes easily at later stages of development.
- **Customer Satisfaction:** The customer sees the product evolve and provides feedback at every loop.

5. Disadvantages

- **Complex:** It is more difficult to manage and understand than the Waterfall model.
- **Expensive:** Risk analysis requires highly specific expertise, which costs money. It is not suitable for small, low-budget projects.
- **Time-Consuming:** The spiral can continue indefinitely if strict controls are not kept (infinite loops).

6. When to use the Spiral Model?

- For **large-scale** systems.
- For **high-risk** projects (e.g., mission-critical software).
- When requirements are complex and likely to change.

Software Project Management

2.0	Software Project Management
2.1	Responsibility of Project Manager
2.2	Project Planning
2.3	Metrics for Project size estimation(LOC and FP)
2.4	Project Estimation Techniques
2.5	COCOMO Models, Basic, Intermediate and complete
2.6	Scheduling
2.7	Organization and Team structure
2.8	Staffing
2.9	Risk Management
2.10	Configuration Management

2.1: Responsibilities of Project Manager

- **Frequency:** Asked **3 Times** (2020, 2023, 2023/24).

• **Question Type:** Usually a **2-mark** question ("What are the responsibilities..."), but you should be prepared to list 5-6 points to ensure full marks.

A **Project Manager (PM)** is the person responsible for leading a project from its inception to execution. This includes planning, execution, and managing the people, resources, and scope of the project.

Key Responsibilities (Memorize these 6 points):

1. **Project Planning:**
 - The PM defines the project's scope (what to build) and objectives.
 - They create the initial plan, deciding "who does what and when".
2. **Project Estimation:**
 - Before coding starts, the PM must estimate the **Cost** (budget) and **Effort** (how many months/people needed) required to finish the project.
 - They use techniques like **COCOMO** or **Function Points** for this.
3. **Scheduling:**
 - The PM breaks the work into smaller tasks and creates a timeline.
 - Tools like **GANTT Charts** or **PERT Charts** are used to track deadlines and ensure the project finishes on time.
4. **Staffing & Team Organization:**
 - The PM recruits the right people (developers, testers, designers).

- They decide the team structure (e.g., Democratic Team vs. Chief Programmer Team).
 - 5. **Risk Management:**
 - The PM must anticipate potential problems (e.g., "What if the server crashes?" or "What if a key developer leaves?").
 - They create a plan to mitigate (avoid or fix) these risks before they destroy the project.
 - 6. **Monitoring & Control:**
 - Once the project starts, the PM tracks progress daily.
 - If the project falls behind schedule, the PM must take corrective action (e.g., adding more staff or cutting non-essential features).
-

Exam Tip: If asked for 2 marks, you can simply write:

"The primary responsibilities of a Project Manager include Project Planning, Cost & Effort Estimation, Scheduling (using Gantt/PERT charts), Staffing the team, and Risk Management to ensure the project is delivered on time and within budget."

2.2: Project Planning.

Exam Status:

- **Frequency:** Asked **1 Time** explicitly ("What is project planning?" in 2020).
- **Relevance:** While rarely asked as a standalone long question, it is the foundation for the "Responsibilities of Project Manager" and "Estimation" questions.

Topic 2.2: Project Planning

1. Definition Project Planning is the systematic process of defining the project's objectives, scope, and the steps required to achieve them *before* the actual development begins. It acts as a roadmap for the development team.

2. Objective The main goal of planning is to ensure that the software is delivered:

- **On Time** (Schedule).
- **Within Budget** (Cost).
- **With High Quality** (Performance).

3. Key Activities in Project Planning When a Project Manager plans a project, they perform these specific tasks:

- **Scope Definition:** clearly defining what the software *will* do (and what it will *not* do) to avoid scope creep.
- **Estimation:** Calculating the resources needed:
 - **Effort Estimation:** How many person-months? (e.g., using COCOMO).

- **Cost Estimation:** How much money?
 - **Resource Estimation:** How many computers, servers, or licenses?
- **Project Scheduling:** Organizing tasks in a timeline (using GANTT or PERT charts) to determine start and end dates.
- **Staffing / Team Structure:** Deciding how many developers are needed and how they will be organized (e.g., Chief Programmer Team).
- **Risk Management:** Identifying potential risks (e.g., "What if the database fails?") and planning solutions.

4. The "Sliding Window" Planning Technique

- In software engineering, we cannot plan the entire 1-year project in perfect detail on Day 1.
- Instead, managers use **Sliding Window Planning**:
 - Plan the **immediate future** (next 2-3 weeks) in *great detail*.
 - Plan the **distant future** (next 6 months) *roughly*.
 - As the project moves forward, the "window" slides, and future tasks are planned in more detail.

Topic 2.3: Metrics for Project Size Estimation (LOC and FP).

Exam Status:

- **Frequency: Very High** (Asked **4 times**: 2020, 2021, 2023, 2024).
- **Question Type:** Usually a **6-mark** or **10-mark** question asking to "Discuss various project size estimation metrics."

Topic 2.3: Project Size Estimation Metrics

To plan a project (budget and schedule), we first need to know "**How big is the software?**" We cannot measure software in kilograms or meters, so we use two specific metrics: **LOC** and **Function Points**.

1. LOC (*Lines of Code*)

This is the simplest and oldest metric. It simply counts the number of lines in the final source code.

- **Unit: KLOC** (Kilo Lines of Code = 1000 lines).
- **Usage:** It is the primary input for the **COCOMO Model**.

Advantages:

- **Simple:** It is very easy to calculate once the code is written.
- **Standard:** Used in many traditional estimation models like COCOMO.

Disadvantages (The "Why it is bad" list):

- **Language Dependent:** This is the biggest flaw.
 - *Example:* A "Hello World" program might take 6 lines in **C** but only 1 line in **Python**. This makes the Python programmer look "lazy" or the project look "smaller," even though the functionality is the same.

- **Programmer Dependent:** An experienced programmer writes concise code (fewer lines), while a beginner might write verbose, messy code (more lines). LOC rewards the messy programmer.
- **Difficult to Estimate Early:** You don't know the exact number of lines *before* you start the project, making it hard to use for early planning.

2. FP (Function Point) Metric

Exam Note: Discussed explicitly in 2024 as a short note.

To solve the problems of LOC, the **Function Point (FP)** metric was developed. Instead of counting lines, it measures the **functionality** provided to the user. It looks at the system from the *user's perspective*.

The 5 Components of Function Point (UFP): To calculate FP, we count five types of features:

1. **External Inputs (EI):** Data entering the system (e.g., "Login screen", "User Registration form").
2. **External Outputs (EO):** Data leaving the system (e.g., "Reports", "Error messages").
3. **External Inquiries (EQ):** Interactive requests (e.g., "Search functionality").
4. **Internal Logical Files (ILF):** Logical data stored inside the system (e.g., "Customer Database").
5. **External Interface Files (EIF):** Data shared with other systems (e.g., sending payment data to a Bank API).

Advantages:

- **Language Independent:** The FP count remains the same whether you use C, Java, or Python because the *functionality* is the same.
- **Better for Planning:** You can estimate Function Points from the **SRS document** (Requirements) before a single line of code is written.

Disadvantages:

- **Complex:** It requires expert judgment to calculate correctly.
- **Subjective:** Two different managers might calculate slightly different FP counts for the same project.

Feature	LOC (Lines of Code)	FP (Function Point)
Focus	Focuses on the <i>Code</i> (Internal).	Focuses on <i>Functionality</i> (User View).
Language	Dependent (Changes with language).	Independent (Constant across languages).
When to use?	Good for existing projects (maintenance).	Good for new projects (early estimation).
Accuracy	Low accuracy for estimation.	Higher accuracy for estimation.

2.4: Project Estimation Techniques.

Exam Status:

- **Frequency: High** (Asked 3 times: 2022, 2023, 2024).
- **Crucial Note:** In exams, the question is almost always phrased as "**Describe Empirical Estimation Techniques.**" (10 Marks). You must focus on the *Empirical* methods (Expert Judgment & Delphi) here.

1. Empirical Estimation Techniques

- **Definition:** These techniques are based on "**Experience**" and "**Educated Guesses.**" They do not rely on complex mathematical formulas but on the human intelligence of people who have done similar projects before.
- **Best used when:** You have limited data and need a quick estimate.

A. Expert Judgment Technique

- **Concept:** A project manager or a domain expert looks at the requirements (SRS) and guesses the cost based on their past experience with similar projects.
- **How it works:**
 - The expert says: *"This looks like the Inventory System we built in 2019. That took 6 months and cost 5 Lakhs. This one has a few more modules, so it will take 7 months."*
- **Advantages:**
 - Fast and cheap to execute.
 - Useful when there is no historical database of past projects.
- **Disadvantages:**
 - **Subjective:** It depends entirely on the expert's mood and memory.
 - **Bias:** The expert might be overly optimistic or pessimistic.

B. Delphi Cost Estimation Technique

- **Concept:** To remove the bias of a single expert, this method uses a **group of experts** and a coordinator to reach a consensus.
 - **Steps:**
 1. **Coordinator:** Provides the problem statement to a group of experts.
 2. **Anonymous Estimation:** Each expert prepares an estimate anonymously (so they aren't influenced by seniors).
 3. **Review:** The coordinator gathers the estimates. If they vary widely (e.g., one says 2 months, another says 10 months), the coordinator asks for reasons.
 4. **Re-Estimation:** Experts review the anonymous reasons and submit a revised estimate.
 5. **Consensus:** The process repeats until the estimates converge to a single number.
-

2. Heuristic Estimation Techniques

- **Definition:** "Heuristic" means a "**Rule of Thumb.**" These techniques use simple mathematical expressions (formulas) derived from historical data (regressions) to predict cost.
- **Relationship:** They assume that Cost is related to one or more independent variables (usually Lines of Code).

A. Single Variable Model

- **Concept:** This model assumes that the Cost/Effort depends on only **one** variable (usually Size/LOC).
- **Formula:** $\text{\$Effort} = c_1 \times (\text{LOC})^{d_1}$
 - Where c_1 and d_1 are constants derived from past project data.
 - *Example:* The **Basic COCOMO Model** is a single-variable heuristic model because it only relies on LOC.

B. Multi-Variable Model

- **Concept:** This assumes that Cost depends on **multiple** variables (Size, Team Skill, Hardware, Complexity, etc.).
- **Formula:** $\text{\$Effort} = c_1 \times (\text{LOC})^{d_1} \times (c_2 \times M)$
 - Where M represents various multipliers (Cost Drivers).
 - *Example:* The **Intermediate COCOMO Model** is a multi-variable heuristic model because it adjusts the result using 15 Cost Drivers (like Reliability, Database Size, Team Experience).

3. Analytical Estimation Techniques

- **Definition:** These techniques derive the estimate from basic theoretical principles and the internal structure of the software, rather than just past data.

A. Halstead's Software Science

- **Concept:** Halstead proposed that just as physical laws govern matter, "Software Science" laws govern code. He measured software based on **tokens** rather than lines.
- **Method:** It counts:
 - n_1 = Number of unique operators (if, while, +, =).
 - n_2 = Number of unique operands (variables like x, y, constants).
 - N_1 = Total occurrences of operators.
 - N_2 = Total occurrences of operands.
- **Formulas:** Using these counts, Halstead provided formulas to calculate:
 - **Volume (V):** How much information is in the code.
 - **Effort (E):** Mental effort required to develop the code.
- **Difference:** Unlike empirical methods (which guess), analytical methods try to "measure" the scientific complexity of the code.

Technique Category	Basis	Examples	Accuracy
Empirical	Human Experience & Judgment	Expert Judgment, Delphi Method	Low (Subjective)
Heuristic	Statistical Relationships (Formulas from Data)	COCOMO (Constructive Cost Model)	High (Data Driven)
Analytical	Theoretical / Scientific Principles	Halstead's Software Science	Medium (Scientific but complex)

2.5: COCOMO Models.

Exam Status:

- **Frequency: Diamond Tier** (Asked **5 times**: 2020, 2021, 2022, 2023, 2024).
- **Significance:** This is the most important topic in Unit 2. You may get a theory question ("Explain COCOMO") or a numerical question ("Calculate the Effort").

2.5: COCOMO (Constructive Cost Model)

1. Definition

COCOMO (Constructive Cost Model) was proposed by **Barry Boehm**. It is an **empirical/heuristic** estimation model used to predict the **Effort** (person-months) and **Development Time** (months) required for a software project based on its size (Lines of Code).

2. The 3 Stages of COCOMO

COCOMO is not a single model but a hierarchy of three models:

1. **Basic COCOMO:** A static, single-variable model. It estimates effort only based on code size (LOC). It is good for quick, early, rough estimates.
2. **Intermediate COCOMO:** A multi-variable model. It starts with the Basic estimate but adjusts it using **15 Cost Drivers** (like software reliability, database size, analyst capability) to get a more accurate result.
3. **Detailed (Complete) COCOMO:** It estimates effort for each *individual module* or subsystem separately rather than the whole project at once. It is the most accurate.

3. The 3 Development Modes (Crucial)

Before using the formula, you must classify your project into one of these three modes:

Feature	Organic Mode	Semi-Detached Mode	Embedded Mode
Project Size	Small (< 50 KLOC)	Medium (< 300 KLOC)	Large (> 300 KLOC)
Complexity	Simple, well-understood.	Intermediate.	Complex, tight constraints.
Team Experience	Highly experienced.	Mixed experience.	Low experience / New to the task.
Example	Simple Payroll System.	Database Management System.	Flight Control System, ATM.

Here are the detailed notes for **Topic 2.5: COCOMO Models**.

Exam Status:

- **Frequency: Diamond Tier** (Asked **5 times**: 2020, 2021, 2022, 2023, 2024).
- **Significance:** This is the most important topic in Unit 2. You may get a theory question ("Explain COCOMO") or a numerical question ("Calculate the Effort")¹¹¹.

Topic 2.5: COCOMO (Constructive Cost Model)

1. Definition

COCOMO (Constructive Cost Model) was proposed by **Barry Boehm**. It is an **empirical/heuristic** estimation model used to predict the **Effort** (person-months) and **Development Time** (months) required for a software project based on its size (Lines of Code)².

2. The 3 Stages of COCOMO

COCOMO is not a single model but a hierarchy of three models³:

1. **Basic COCOMO:** A static, single-variable model. It estimates effort only based on code size (LOC). It is good for quick, early, rough estimates.
 2. **Intermediate COCOMO:** A multi-variable model. It starts with the Basic estimate but adjusts it using **15 Cost Drivers** (like software reliability, database size, analyst capability) to get a more accurate result.
 3. **Detailed (Complete) COCOMO:** It estimates effort for each *individual module* or subsystem separately rather than the whole project at once. It is the most accurate.
-

3. The 3 Development Modes (Crucial)

Before using the formula, you must classify your project into one of these three modes⁴:

Feature	Organic Mode	Semi-Detached Mode	Embedded Mode
Project Size	Small (< 50 KLOC)	Medium (< 300 KLOC)	Large (> 300 KLOC)
Complexity	Simple, well-understood.	Intermediate.	Complex, tight constraints.
Team Experience	Highly experienced.	Mixed experience.	Low experience / New to the task.
Example	Simple Payroll System.	Database Management System.	Flight Control System, ATM.

4. Basic COCOMO Formulas

To solve numerical problems in the exam, memorize these two formulas and the table of constants.

Formulas:

- Effort (E)** $E = a \times (\text{KLOC})^b$
 - Result is in Person-Months (PM).
- Development Time (D)** $D = c \times (\text{Effort})^d$
 - Result is in Months.

Table of Constants:

(You must memorize these values)

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38

Mode	a	b	c	d
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

5. Numerical Example (Based on 2021 Exam Question)

Question: Suppose a project was estimated to be **400 KLOC**. Calculate the Effort and Development Time for the **Embedded** model⁵.

Solution:

1. **Identify Mode:** Embedded.
2. **Identify Constants:** \$a = 3.6\$, \$b = 1.20\$, \$c = 2.5\$, \$d = 0.32\$.
3. Calculate Effort (E):

$$E = 3.6 \times (400)^{1.20}$$

(Using calculator: $400^{1.2} \approx 1320$)

$$E = 3.6 \times 1320 = 4752 \text{ Person-Months}$$

4. Calculate Time (D):

$$D = 2.5 \times (4752)^{0.32}$$

(Using calculator: $4752^{0.32} \approx 14.8$)

$$D = 2.5 \times 14.8 = 37 \text{ Months}$$

Answer: The project will require approximately **4752 person-months** of work and will take **37 months** to complete.

2.6: Scheduling.

Exam Status:

- **Frequency: High** (Asked 4 times: 2020, 2022, 2023, 2024).
- **Key Questions:** "What is Project Scheduling?", "Why is Gantt Chart used?", "Define PERT", and "Define Critical Path."

2.6: Project Scheduling

1. Definition

Project Scheduling is the process of breaking down the software project into smaller tasks (activities) and assigning a **start date** and **end date** to each task. It ensures that the project is completed on time and resources (developers) are used efficiently.

- **Goal:** To answer "Who does what and by when?"

2. Why do we need Scheduling?

- To identify task dependencies (e.g., "Coding cannot start until Design is finished").
- To track progress (Are we ahead or behind schedule?).
- To identifying the **Critical Path** (the most important tasks).

3. Scheduling Tools (The Exam Questions)

Software Project Managers use two main tools to schedule projects: **Gantt Charts** and **PERT Charts**.

A. Gantt Chart

(Asked in 2022 & 2024: "Why is Gantt Chart used?")

- **Definition:** A Gantt chart is a horizontal **bar chart** that represents the project schedule.
- **Structure:**
 - **X-axis (Horizontal):** Represents **Time** (Days/Weeks).
 - **Y-axis (Vertical):** Represents the list of **Tasks/Activities**.
 - **Bars:** The length of the bar shows the duration of the task.
- **Why is it used? (Exam Answer):**
 1. It provides a quick visual timeline of the project.
 2. It shows the **Start Date** and **End Date** of every task.
 3. It helps managers track "Planned Progress" vs. "Actual Progress."
- **Disadvantage:** It does not clearly show the *dependencies* between tasks (i.e., it's hard to see which task waits for which).

B. PERT Chart (Program Evaluation Review Technique)

(Asked in 2024: "Define PERT. Explain why PERT charts are used.")

- **Definition:** PERT is a **Network Diagram** (graph) used to analyze the tasks involved in completing a project. It focuses on the time required to complete each task.
- **Structure:** It consists of **Nodes** (circles representing events) and **Arrows** (lines representing tasks).
- **Why is it used?**
 1. It clearly shows the **dependencies** between tasks (e.g., Task B cannot start until Task A ends).
 2. It allows the calculation of the **Critical Path**.
 3. It handles uncertainty in time estimates better than Gantt charts.

C. Critical Path Method (CPM)

(Asked in 2024: "Define Critical Path.")

- **Definition:** The Critical Path is the **longest path** through the project network diagram (PERT chart).
- **Significance:**
 - It determines the **shortest possible time** to complete the entire project.
 - **Zero Slack:** Tasks on the critical path have "zero slack" (or float). This means if a critical task is delayed by even 1 day, the **entire project** will be delayed by 1 day.
 - **Management Focus:** Project managers must monitor critical path tasks most closely.

Feature	Gantt Chart	PERT Chart
Visual Style	Bar Chart	Network Diagram (Graph)
Best For	Tracking Progress & Timelines	Analyzing Dependencies & Critical Path
Clarity	Easy to read for beginners	Complex to read
Dependencies	Hard to see	Very clear

2.7: Team Structure.

Exam Status:

- **Frequency:** Asked **2 times** (2021, 2024).
- **Question Type:** "Explain team structure with diagram." (6 Marks or 5 Marks)

2.7: Team Structure

Definition:

Team structure refers to the way individual team members are organized to work on a software project. The goal is to maximize productivity, communication, and morale while minimizing conflict.

There are three main types of team structures:

1. Democratic Team (Egoless Team)

- **Structure:** There is **no permanent leader**. Decisions are made by group consensus. Leadership rotates depending on the task (e.g., the best coder leads coding; the best designer leads design).
- **Philosophy:** "Egoless programming" – code belongs to the team, not the individual.

- **Pros:** High morale, excellent for research/difficult problems, innovative.
- **Cons:** Slow decision-making (lots of meetings), lack of authority can lead to chaos.

2. Chief Programmer Team (Controlled Centralized)

- ****Structure:**** modeled after a surgical team. There is one **Chief Programmer** who makes all key decisions and does the critical work.
 - **The Chief:** The "Star" who designs and codes the core.
 - **Backup Programmer:** Ready to take over if the Chief leaves.
 - **Librarian:** Handles documentation and administration.
 - **Programmers:** Support the Chief.
- **Pros:** Fast decision-making, clear authority, efficient for routine tasks.
- **Cons:** Low morale for lower-level programmers (they feel like assistants), single point of failure (if the Chief quits, the project dies).

3. Hierarchical (Mixed) Team

- **Structure:** A mix of both. It has a Project Manager at the top, followed by Senior Engineers (Team Leads), and then Junior Engineers.
- **How it works:** This is the standard corporate structure. The manager handles administration, leads handle technical parts, and juniors execute tasks.
- **Pros:** Scales well for very large projects, clear career path.
- **Cons:** Communication overhead can be high between layers.

Exam Tip: Since the exam asks for a **diagram**, draw a simple hierarchy:

- **Democratic:** A circle with nodes connected to everyone else (Mesh).
- **Chief Programmer:** A star shape with the Chief in the center.
- **Hierarchical:** A standard tree structure (Manager $\rightarrow$ Leads $\rightarrow$ Devs).

3.0 Requirement Analysis and specification

3.1: Requirements Gathering and Analysis

1. Definition

Requirement Engineering is the process of understanding the customer's needs and documenting them. It is the first and most critical phase of software development. If you get the requirements wrong, the whole project fails.

It consists of two main activities:

1. **Gathering (Elicitation):** Collecting requirements from users, customers, and stakeholders.
2. **Analysis:** Studying the gathered requirements to identify inconsistencies, incompleteness, or feasibility issues.

3.2: Software Requirements Specification (SRS).

Exam Status:

- **Frequency: Diamond Tier** (Asked **5 times** in 5 years: 2020, 2021, 2022, 2023, 2024).
- **Significance:** This is the most consistent question in the entire syllabus. You are almost guaranteed to see a question asking for "Characteristics of a Good SRS" (10 marks) or "Why it is called Black-Box" (2 marks).

Topic 3.2: Software Requirements Specification (SRS)

1. Definition

An **SRS (Software Requirements Specification)** is a formal document that describes the **nature** of the software to be developed.

- It serves as the **final agreement (contract)** between the **Customer** (who wants the software) and the **Developer** (who builds it).
- It describes **"WHAT"** the system should do, but it does **NOT** describe **"HOW"** it should do it.
- It acts as the foundation for the Design and Testing phases.

2. Why is SRS called a "Black-Box Specification"?

(Asked in 2021 and 2022) Why is SRS called a "Black-Box Specification"?

The SRS (Software Requirement Specification) is called a **"Black-Box Specification"** because it describes the system entirely from the **outside view** (the user's perspective) without revealing anything about the **inside view** (the developer's perspective).

Key Reasons:

1. **"What" vs. "How":**
 - The SRS defines **WHAT** the system must do (functionality, inputs, and outputs).
 - It deliberately hides **HOW** the system achieves it (internal logic, code, algorithms, or data structures).
2. **External Visibility:**
 - Like a sealed black box, the user can only see the **Inputs** going in and the **Outputs** coming out. The internal mechanism remains invisible.
 - *Example:* An SRS for a calculator says, "If the user inputs $2 + 2$, the system shall display 4." It does *not* explain how the binary addition happens inside the CPU.
3. **Independence from Implementation:**
 - By treating the system as a black box, the SRS ensures the requirements are **implementation-independent**. This allows developers to change the internal technology (e.g., switching from Java to Python) without changing the requirements.

Topic 3.2.2: Characteristics of a Good SRS

(Crucial 10-Mark Question)

A "Good" SRS must satisfy specific criteria to ensure the project doesn't fail. IEEE defines these characteristics:

1. **Unambiguous (Most Important):**
 - Every requirement must have **only one interpretation**.
 - *Example:* Instead of saying "The system should be fast" (vague), say "The system shall respond within 2 seconds."
2. **Correct:**
 - The SRS should accurately reflect the actual needs of the customer. If the document says "Red" but the customer wants "Blue," it is incorrect.
3. **Complete:**
 - It must describe **everything** the software needs to do. No functionality should be left out ("TBD" or "To Be Determined" is not allowed).
4. **Consistent:**
 - There should be no conflicts between requirements.
 - *Example of Inconsistency:* Requirement A says "All users must have a password," but Requirement B says "Guest users can login without a password."
5. **Verifiable (Testable):**
 - A requirement is verifiable only if there is a cost-effective way to **test** it.
 - *Bad:* "The software should be user-friendly." (Subjective/Not testable).
 - *Good:* "A new user should be able to create an account in under 3 minutes."
6. **Traceable:**
 - Each requirement should have a unique ID (e.g., REQ-001). This allows developers to trace the requirement to the specific code that implements it, and testers to the specific test case that verifies it.
7. **Modifiable:**
 - The structure and style of the SRS should allow for easy changes to requirements without breaking the document's consistency.

Topic 3.2.1: Contents of SRS

(Asked in 2020 and 2024)

What is written inside the document? The standard IEEE structure typically includes:

1. Introduction:

- **Purpose:** What is the objective of this software?
- **Scope:** What will the software do (and what will it NOT do)?
- **Definitions:** Explaining technical terms.

2. General Description:

- **Product Perspective:** Is this a new product or part of a larger system?
- **User Characteristics:** Who will use it? (Experts, general public, etc.)
- **Assumptions:** e.g., "We assume the user has internet access."

3. Specific Requirements (The Core):

- **Functional Requirements:** Describes the specific behaviors/functions.
 - *Example:* "The system shall calculate 18% GST on all invoices."
- **Non-Functional Requirements:** Quality attributes.
 - **Performance:** Speed, latency.
 - **Reliability:** Uptime, failure rate.
 - **Security:** Encryption, access levels.
- **External Interface Requirements:**
 - Hardware interfaces (Printers, Sensors).
 - Software interfaces (Operating System, Database).

Topic 3.2.3: Organization of SRS

The SRS is usually organized effectively to handle complexity.

- It is often written in **Natural Language** (English) so customers can understand it.
- However, for complex logic, natural language can be confusing.

Topic 3.2.4: Techniques for Representing Complex Logic

(Briefly mentioned in syllabus) To avoid ambiguity in complex logic, we use formal techniques instead of just English text:

1. **Decision Trees:** A graphical tree to represent choices and outcomes.
2. **Decision Tables:** A table showing conditions and actions.
3. **Structured English:** Using limited vocabulary (IF-THEN-ELSE) to describe logic clearly.

Topic	Frequency	Question Type	Priority
Cohesion & Coupling	5 Times	Define & Classify (10 Marks), Differentiate (5 Marks)	 Diamond
Data Flow Diagrams (DFD)	5 Times	Draw DFD for X, Symbols, Levels (10 Marks)	 Diamond

Topic	Frequency	Question Type	Priority
Fan-in / Fan-out	3 Times	Definition (2 Marks)	 Silver
Structure Analysis	2 Times	Definition, Transform vs Transaction Analysis	 Silver

UNIT 4: Software Design

4.1: What is a Good Software Design?

1. Definition

Software Design is the process of transforming the user requirements (from the SRS) into a suitable form (blueprint) that helps the programmer in coding and implementation. A "Good" design ensures the system is built correctly and is easy to maintain.

2. Characteristics of a Good Software Design

For a software design to be considered "Good," it must possess the following attributes:

- Correctness:**
 - The design must correctly implement all the requirements specified in the SRS document.
- Understandability:**
 - The design should be easy to read and understand. If a new developer joins the team, they should be able to understand the architecture without needing the original designer.
- Efficiency:**
 - The design should use resources (CPU, Memory) efficiently.
- Maintainability:**
 - It should be easy to change or fix. This is the most important characteristic because maintenance costs the most money.
- Modularity:**
 - The system should be broken down into smaller, independent modules (functions/subroutines) rather than being one giant block of code.

3. The "Golden Rule" of Good Design

(This is the technical answer expected in exams)

A good software design **must** achieve:

- High Cohesion:** Each module should do **one thing** and do it well (Internal strength).
- Low Coupling:** Modules should depend on each other as little as possible (Independence).

Note: If you write only one thing, write: *"A good design has High Cohesion and Low Coupling."*

4. Software Design Approaches

(Asked in 2020: "Name two approaches of software design.")

There are two main strategies to design a system:

1. Top-Down Approach:

- You start with the main system (the "Top") and break it down into smaller subsystems. Then break those into even smaller modules.
- *Analogy:* Designing a car by first drawing the whole car, then the engine, then the pistons.
- *Used in:* Structural Programming (C, Pascal).

2. Bottom-Up Approach:

- You start by designing the smallest components (basic modules) first and then combine them to form larger subsystems, eventually forming the complete system.
- *Analogy:* Building a wall by laying one brick at a time.
- *Used in:* Object-Oriented Programming (C++, Java).

4.2: Cohesion and Coupling.

Exam Status:

- **Frequency: Diamond Tier** (Asked **5 times**: 2020, 2021, 2022, 2023, 2024)
 - **Question Type:** "Define cohesion and coupling. Explain their classification." (10 Marks) or "Differentiate between cohesion and coupling." (5 Marks).
 - **Golden Rule:** A good software design always aims for **High Cohesion** and **Low Coupling**.
-

A. Cohesion (Internal Strength)

1. Definition

Cohesion is a measure of the functional strength of a module. It measures how closely the elements (instructions/logic) inside a single module are related to each other.

- **Goal: High Cohesion.** We want a module to focus on **one single task** and do it well.

2. Classification of Cohesion (From Best to Worst)

You must list these in order to get full marks.

1. Functional Cohesion (Best):

- The module performs exactly **one** specific, well-defined task.
- *Example:* `CalculateSquareRoot()`, `ComputeTax()`.

2. Sequential Cohesion:

- The output of one part of the module becomes the input for the next part. Like an assembly line.

- *Example:* `FormatData()` $\rightarrow$ `ValidateData()` (inside one module).
 - 3. **Communicational Cohesion:**
 - All elements operate on the **same input data** or produce the same output data.
 - *Example:* A module that prints a student record AND saves the student record to a file (both use the "Student Record").
 - 4. **Procedural Cohesion:**
 - Elements are grouped because they must be executed in a specific order (sequence) to achieve a result, often following a flowchart.
 - *Example:* A function that loops through steps 1, 2, and 3 of an algorithm.
 - 5. **Temporal Cohesion:**
 - Elements are grouped because they are executed at the **same time** (time-based).
 - *Example:* `SystemInitialization()` (It turns on the printer, opens the database, and clears RAM—unrelated tasks, but all happen at startup).
 - 6. **Logical Cohesion:**
 - Elements are logically related types of tasks, but not functionally related.
 - *Example:* A single `Input()` module that handles keyboard input, mouse input, and mic input. (Ideally, these should be separate).
 - 7. **Coincidental Cohesion (Worst):**
 - Elements are grouped arbitrarily with no meaningful relationship.
 - *Example:* A "Utils" file containing random functions like `PrintPage()` and `CalculateDate()`.
-

B. Coupling (External Dependency)

1. Definition

Coupling is a measure of the interdependence between different modules. It measures how much one module relies on the internal details of another.

- **Goal: Low Coupling.** Modules should be independent. If you change Module A, it should not break Module B.

2. Classification of Coupling (From Best to Worst)

1. **Data Coupling (Best/Lowest):**
 - Modules communicate by passing only the necessary data items (simple parameters like integers or strings).
 - *Example:* Module A calls `Sqrt(x)`. It only passes `x`.
2. **Stamp Coupling:**
 - Modules share a composite data structure (like a `Record` or `Object`), even if they only need one field from it.
 - *Example:* Passing the entire `EmployeeRecord` to a function that only needs the `EmployeeID`.
3. **Control Coupling:**
 - One module passes a "Control Flag" to another module, effectively telling it what logic to execute.
 - *Example:* Module A sends a boolean `True` to Module B to say "Do the error logic." (This is bad because A controls B's internals).
4. **Common Coupling:**

- Multiple modules share **Global Data** (global variables).
- *Why it's bad:* If one module corrupts the global variable, all other modules using it will fail. It is very hard to debug.

5. Content Coupling (Worst/Highest):

- One module directly accesses or modifies the internal data or code of another module.
- *Example:* Module A using a `GOTO` statement to jump into the middle of Module B.

Difference between Cohesion and Coupling

(Use this table for 5-mark differentiation questions)

Feature	Cohesion	Coupling
Focus	Intra-module (Inside one module).	Inter-module (Between two modules).
Definition	Measure of functional strength.	Measure of dependency.
Relationship	Elements within the module.	Relationship between modules.
Goal	Maximize (High Cohesion).	Minimize (Low Coupling).
Design Principle	"Do one thing well."	"Don't rely on others."
Best Type	Functional Cohesion.	Data Coupling.
Worst Type	Coincidental Cohesion.	Content Coupling.

4.3: Neat Arrangement (Modularity & Structure Principles).

Exam Status:

- **Syllabus Context:** "Neat Arrangement" refers to the organization of modules in a design. In exams, this concept is tested via specific metrics like **Fan-in** and **Fan-out**.
- **Frequency:** **Fan-in** and **Fan-out** have been asked **3 Times** (2021, 2022, 2024) as 2-mark definitions.
- **Significance:** While "Neat arrangement" itself isn't asked, the *metrics* that measure a neat arrangement (Fan-in/Fan-out) are important.

4.3: Neat Arrangement (Modularity & Design Metrics)

To ensure a software design is "neatly arranged," we use the principle of **Modularity**. This means breaking the large system into smaller, manageable, and independent pieces called **Modules**.

To measure if the modules are arranged "neatly" (effectively), we use two key metrics: **Fan-in** and **Fan-out**.

1. Fan-in

(Asked in 2021, 2022, 2024)

- **Definition:** Fan-in is the number of **super-ordinate** modules that call (invoke) a specific module.
- **Visual:** Count the number of arrows pointing **INTO** the module bubble in a Structure Chart.
- **Good or Bad?**
 - **High Fan-in is GOOD.**
 - **Why?** If many modules call Module X, it means Module X is a useful utility function (like CalculateTax or PrintPage) that is being **reused** across the system. This reduces duplicate code.

2. Fan-out

(Asked in 2021, 2022, 2024)

- **Definition:** Fan-out is the number of **sub-ordinate** modules that are called (invoked) by a specific module.
- **Visual:** Count the number of arrows pointing **OUT** of the module bubble.
- **Good or Bad?**
 - **High Fan-out is BAD.**
 - **Why?** If Module X calls 20 other modules, it is doing too much work (coordinating too many things). It is too complex and hard to maintain. A good design usually limits Fan-out to 5–7 (known as the **Span of Control**).

Metric	Fan-in	Fan-out
Definition	Count of modules <i>calling</i> this module.	Count of modules <i>called by</i> this module.
Goal	High is desirable (Reuse).	Low is desirable (Simplicity).
Indicates	Code Reusability.	Complexity / Span of Control.

4.4: Software Design Approaches.

Exam Status:

- **Frequency:** Asked **1 time** (2020: "Name two approaches of software design").

- **Clarification:** You asked for "System Oriented," but in standard Software Engineering (and your syllabus), this is technically called **Function-Oriented Design** (or Structured Design). The two competing approaches are **Function-Oriented** vs. **Object-Oriented**.

Topic 4.4: Software Design Approaches

There are two fundamental ways to design a software system.

1. Function-Oriented Design (System-Oriented / Structured)

- **Focus:** This approach focuses on the **Functions** (processes/algorithms) of the system. It asks, "*What does the system DO?*"
- **Structure:** The system is viewed as a set of interacting functions. The main function is broken down into smaller sub-functions (Top-Down decomposition).
- **Data Handling:** It treats **Data** and **Functions** separately. Global data is often shared between functions, which can lead to issues (High Coupling).
- **Approach: Top-Down Approach.** You start with the high-level description and break it down.
- **Example Languages:** C, Pascal, FORTRAN.
- **Diagram Tool: Data Flow Diagrams (DFD)** are the primary tool for this approach.

2. Object-Oriented Design (OOD)

- **Focus:** This approach focuses on the **Objects** (real-world entities) involved in the system. It asks, "*What items are in the system?*" (e.g., Student, Teacher, Course).
- **Structure:** The system is viewed as a collection of interacting objects. Each object contains its own data and the functions that operate on that data.
- **Data Handling:** It combines **Data** and **Functions** into a single unit called a Class (**Encapsulation**). This protects the data from accidental corruption.
- **Approach: Bottom-Up Approach.** You design the basic components (Objects) first and then assemble them to form the system.
- **Example Languages:** Java, C++, Python.
- **Diagram Tool:** UML (Unified Modeling Language) Class Diagrams.

Feature	Function-Oriented (System)	Object-Oriented
Primary Focus	Logic / Algorithms (Functions).	Real-world Entities (Objects).
Design Strategy	Top-Down (Decomposition).	Bottom-Up (Composition).
Data & Code	Separate (Global data is common).	Combined (Encapsulation).
Reusability	Low (Hard to reuse functions).	High (Classes are easily reused).
Maintenance	Difficult for large systems.	Easier (due to Modularity).
Example	C Programming.	Java / C++ Programming.

4.5: Structured Analysis.

Exam Status:

- **Frequency: Medium.**
- **Recent Questions:**
 - **2024:** "Define structure analysis." (2 Marks) ¹.
 - **2023:** "Explain Transform Analysis and Transaction Analysis." (6 Marks) ².

Topic 4.5: Structured Analysis

1. Definition

Structured Analysis is a **top-down** software engineering technique used to analyze and specify the requirements of a system. It focuses on the **flow of data** and the **functions** (processes) that transform that data.

- It creates a graphical representation of the system (using DFDs) which is easier for users and developers to understand than plain text.
- **Goal:** To convert the messy "Requirements" into a clear logical model that can be designed and coded.

2. Tools of Structured Analysis

To perform structured analysis, we use the following tools:

1. **Data Flow Diagrams (DFD):** Shows how data moves and changes. (Covered in 4.6).
2. **Data Dictionary:** A centralized repository that defines the structure of all data elements (e.g., "What is a 'Student Name'? It is a string of 30 chars").
3. **Decision Trees / Decision Tables:** Used to represent complex logic inside a process.

Topic 4.13: Transform Analysis vs. Transaction Analysis

(Important: Asked as a 6-mark question in 2023)

After drawing the DFD, we need to convert it into a **Structure Chart** (a hierarchy of modules) for the final design. There are two strategies to do this conversion:

A. Transform Analysis

- **When to use:** Use this when the system follows a clear **Input $\rightarrow$ Process $\rightarrow$ Output** flow. The data enters, gets "transformed" (processed), and leaves.
- **How it works:**
 1. Identify the **Afferent Branch** (Input path).
 2. Identify the **Efferent Branch** (Output path).
 3. Identify the **Central Transform** (The main work/logic in the middle).
 4. Create a "Main Module" that controls these three parts.

B. Transaction Analysis

- **When to use:** Use this when the system splits into multiple distinct paths based on a user's choice (a "Transaction").
- **Analogy:** A menu system (e.g., "Press 1 for Withdrawal, Press 2 for Deposit").
- **How it works:**
 1. Identify the **Transaction Center** (the bubble that splits the flow).
 2. Create a "Dispatcher" module that looks at the transaction type and calls the correct sub-module (e.g., Call Withdrawal Module OR Call Deposit Module).

Feature	Transform Analysis	Transaction Analysis
Data Flow	Sequential (Input $\rightarrow$ Process $\rightarrow$ Output).	Parallel / Conditional (Choice based).
Structure	Linear flow.	Radiating flow (Fan-out).
Main Component	Central Transform (Logic).	Transaction Center (Dispatcher).
Example	A calculator (Input numbers $\rightarrow$ Add $\rightarrow$ Show result).	An ATM Menu (Choose Withdraw / Balance / Deposit).

4.6: Data Flow Diagrams (DFD).

Exam Status:

- **Frequency:** **Diamond Tier** (Asked **5 times**: 2020, 2021, 2022, 2023, 2024).
- **Question Type:** "Define DFD. Explain different symbols used. Discuss shortcomings." (10 Marks) or "Draw DFD for [specific system]" (10 Marks).

Topic 4.6: Data Flow Diagrams (DFD)

1. Definition

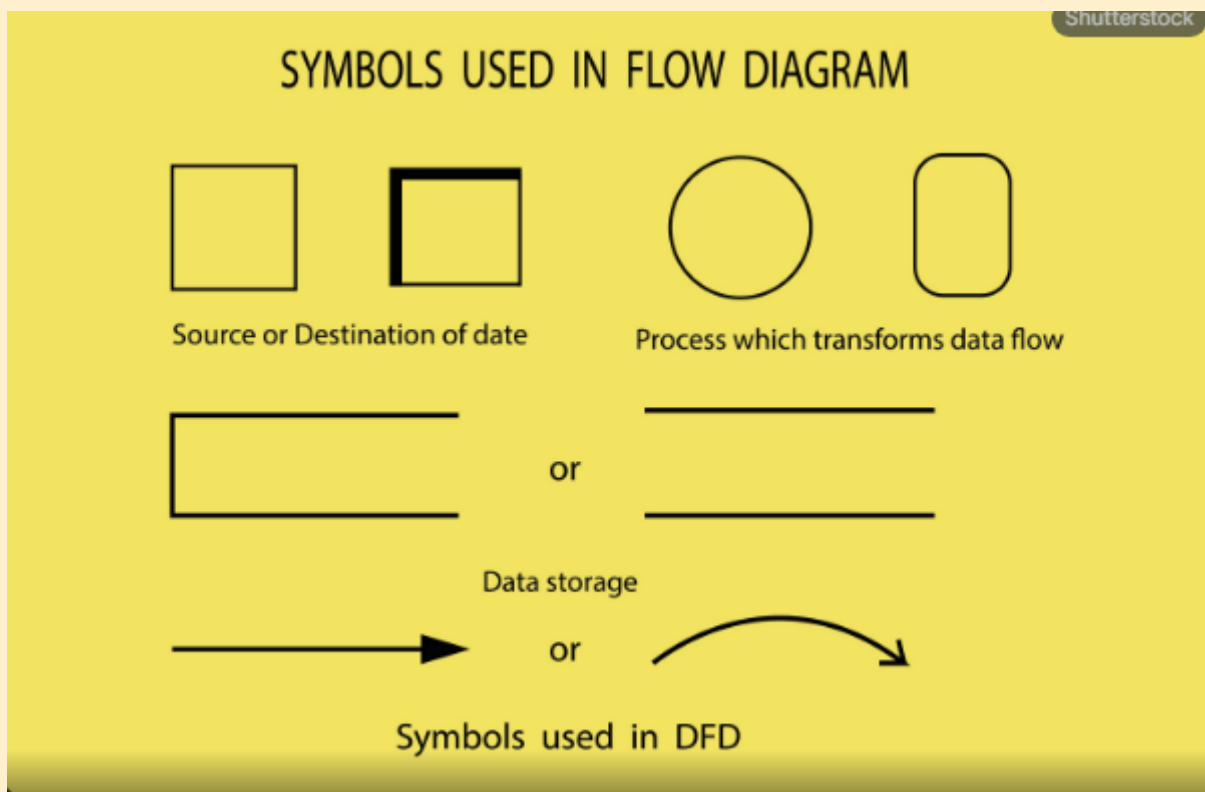
A **Data Flow Diagram (DFD)** is a graphical representation of the flow of data through an information system. It shows:

- Where data comes from (**Source**).
- Where data goes (**Destination**).
- How data is processed (**Process**).
- Where data is stored (**Data Store**).

- **Key Concept:** DFDs depict the **flow of data**, NOT the flow of control. It does not show loops (for, while) or conditions (if-else). It is a hierarchical model (Top-Down).

2. Symbols Used in DFD

Component	Symbol (Standard Gane & Sarson or Yourdon)	Description
External Entity	Rectangle / Square	Represents a source or destination of data outside the system (e.g., User, Admin, Sensor, Customer).
Process	Circle / Bubble	Represents a function or action that transforms input data into output data (e.g., "Calculate Tax", "Verify Login").
Data Store	Open-ended Rectangle	Represents a place where data is saved for later use (e.g., Database, File, Filing Cabinet).
Data Flow	Arrow (\$\rightarrow\$)	Represents the movement of data between entities, processes, and stores. The arrow label names the data moving.



Levels of DFD

In Software Engineering, we cannot draw the entire complex system in one single diagram because it would be too messy. Instead, we use a **Top-Down Approach** (Hierarchical). We start with a high-level overview and then "explode" (break down) each part into more detail.

This "Explosion" of processes creates different **Levels of DFD**.

1. Level 0 DFD: The Context Diagram

(Also known as the "Bird's Eye View")

- **Purpose:** To define the **boundary** of the system. It shows how the system interacts with the outside world.
- **Key Feature:** The entire system is represented as **ONE single Process bubble** (Circle).
- **Components Visible:**
 - **External Entities:** (Who interacts with the system? e.g., User, Admin).
 - **Input/Output Flows:** (What data goes in and what comes out?).
- **What is Hidden:**
 - NO Internal processes.
 - NO Data stores (Files/Databases are usually hidden at this level unless they are external).

Example (Library System):

- **Bubble:** "Library Management System"
- **Entities:** "Student" and "Librarian".
- **Flow:** Student sends "Book Request" $\rightarrow$ System $\rightarrow$ sends "Book Status".

2. Level 1 DFD: System Overview

(The "Zoomed In" View)

- **Purpose:** To break the single "System Bubble" from Level 0 into its **major functional modules**.
- **Key Feature:** The single bubble is exploded into **3 to 7 smaller bubbles** representing the main functions.
- **Components Visible:**
 - **Internal Processes:** Major functions (e.g., "Issue Book", "Return Book", "Calculate Fine").
 - **Data Stores:** This is the level where files/databases usually appear (e.g., "Student File", "Book File").
 - **Data Flows:** Internal movement of data between these processes and files.
- **Balancing Rule:** The inputs and outputs coming from external entities **must match** exactly with Level 0.

Example (Library System Level 1):

- Process 1: **Register Student**.
- Process 2: **Issue Book**.

- Process 3: **Return Book.**
- Data Store: **Book Database.**

3. Level 2 DFD: Detailed View

(The "Micro" View)

- **Purpose:** To break down **one specific process** from Level 1 into more detailed sub-processes.
- **Key Feature:** If Process 2 ("Issue Book") in Level 1 is complex, we draw a Level 2 DFD just for that bubble.
- **When to stop?** We continue exploring levels (Level 3, Level 4...) until a process is simple enough to be described in a few lines of code or a short paragraph.

Example (Exploding "Process 2: Issue Book"):

- Sub-Process 2.1: **Check User Validity.**
- Sub-Process 2.2: **Check Book Availability.**
- Sub-Process 2.3: **Update Book Status.**

Feature	Level 0 (Context)	Level 1	Level 2
Complexity	Very Simple (Abstract).	Basic Overview.	High Detail.
Processes	Only 1 Process.	3-7 Major Processes.	Detailed Sub-processes.
Data Stores	Usually Hidden.	Visible.	Visible and specific.
Audience	Customers / Stakeholders.	Project Managers / Analysts.	Programmers / Designers.

4.11: Structured Design

1. Definition

Structured Design is a systematic approach to software design that focuses on breaking the system down into a hierarchy of independent modules.

- **Goal:** To convert the **Data Flow Diagram (DFD)** created during analysis into a **Structure Chart** (a roadmap for programmers).
- **Key Principle:** "Divide and Conquer." It solves a complex problem by dividing it into smaller, manageable pieces (modules).

2. The Main Tool: Structure Chart

While Structured *Analysis* uses DFDs, Structured *Design* uses **Structure Charts**.

- **What is a Structure Chart?**
 - It is a hierarchical diagram that shows the **modules** of a system and how they communicate.
 - Unlike DFDs (which show data flow), Structure Charts show **Control Flow** (which module calls which).
- **Components:**
 - **Module:** A rectangle representing a function/subroutine.
 - **Connection:** An arrow showing one module calling another.
 - **Couple:** Small arrows with circular tails showing data passing between modules (Data Couple vs Control Couple).

3. Objectives of Structured Design

A successful structured design must achieve the "Gold Standard" of software engineering (which we covered in 4.2):

1. **High Cohesion:** Each module should focus on a single task.
2. **Low Coupling:** Modules should be independent and interact minimally.
3. **Factoring:** The process of breaking a large module into smaller sub-modules to reduce size and complexity.

4. Structured Design Methodology (The Process)

The transition from Analysis to Design happens in these steps:

1. **Review the DFD:** Ensure the Data Flow Diagram (from Topic 4.6) is correct.
2. **Identify the Flow Type:** Determine if the system is **Transform-centered** (linear input-process-output) or **Transaction-centered** (menu/choice based).
3. **Map to Structure Chart:**
 - Convert DFD bubbles into Structure Chart modules.
 - Use **Transform Analysis** or **Transaction Analysis** (Topic 4.13) to arrange them hierarchically.
4. **Refine the Design:** Use heuristics (Fan-in/Fan-out, Coupling checks) to improve the quality.

4.12: Principles of Transformation of DFD to Structure Chart.

Exam Status:

- **Frequency:** **Low** as a direct standalone question, but it is the **mandatory step** to understand Transform/Transaction Analysis (Topic 4.13), which was asked in 2023.
- **Concept:** This topic explains the rules for converting the analysis model (DFD) into the design model (Structure Chart).

Topic 4.12: Transformation of DFD to Structure Chart

1. The Goal

We have a **Data Flow Diagram (DFD)** from the Analysis phase, which shows *how data moves*. We need to convert it into a **Structure Chart (SC)** for the Design phase, which shows *how modules call each other* (Control Flow).

- **DFD:** Network of bubbles (processes).
- **Structure Chart:** Hierarchy of boxes (modules).

2. Two Types of Data Flow

To perform this transformation, we first look at the DFD and decide which "shape" the data flow has. There are two principles based on the flow type:

A. Transform Flow (Sequential)

- Data enters, gets processed in a linear sequence, and leaves.
- *Structure:* Input $\rightarrow$ [Transform] $\rightarrow$ Output.
- *Transformation Principle:* **Transform Analysis**.

B. Transaction Flow (Divergent)

- Data enters and triggers one of many distinct paths based on a "choice" or "transaction type."
- *Structure:* Input $\rightarrow$ [Switch/Decision] $\rightarrow$ Path A / Path B / Path C.
- *Transformation Principle:* **Transaction Analysis**.

3. Principles of Transform Analysis (The Steps)

Used when the DFD looks like a linear pipeline.

1. **Identify the Main Data Path:** Trace the data from input to output.
2. **Identify Boundaries:** Draw lines on the DFD to separate it into three parts:
 - **Afferent Branch (Input):** Processes that prepare/validate input.
 - **Central Transform (Processing):** The core work/logic.
 - **Efferent Branch (Output):** Processes that format/print output.
3. **Create the Structure Chart Hierarchy:**
 - Create a **Root Module** (Main Controller).
 - Under the Root, create three subordinate modules:
 1. **Input Controller** (Get Data).
 2. **Process Controller** (Do Work).
 3. **Output Controller** (Send Data).
4. **Factoring:** Break down these controllers into the specific bubbles found in the DFD.

4. Principles of Transaction Analysis (The Steps)

Used when the DFD looks like a starburst or menu.

1. **Identify the Transaction Center:** Find the bubble where the data splits into different paths (e.g., "Determine User Choice").
2. **Create the Dispatcher Module:**
 - Create a **Root Module** (Dispatcher) that evaluates the transaction.
3. **Create Subordinate Modules:**
 - For every possible path in the DFD, create a module in the Structure Chart.
 - *Example:* If the user can choose "Add," "Edit," or "Delete," the Dispatcher will call the Add Module, Edit Module, or Delete Module.

5. Factoring (Refining the Design)

Once the basic transformation is done, we apply the principle of **Factoring**:

- **First-Level Factoring:** Identifying the main modules (Input, Process, Output).
- **Second-Level Factoring:** Breaking large modules into smaller ones to ensure **High Cohesion** and **Low Coupling**.

UNIT 5: User Interface Design.

Topic	Frequency	Question Type	Priority
Types of User Interfaces	5 Times	Classify UI, CLI vs GUI, Menu-based	 Diamond
Characteristics of Good UI	3 Times	"Write characteristics of good interface"	 Gold
Menu Types	2 Times	"List different menus used"	 Silver
UID Metaphor	1 Time	"What is a metaphor?"	 Silver

5.1: Characteristics of a Good Interface.

Exam Status:

- **Frequency: Gold Tier** (Asked **3 times**: 2020, 2023, 2024).
- **Question Type:** "Explain the characteristics of a good interface" (10 Marks in 2020) or "Write down characteristics" (2 Marks in 2023).

Topic 5.1: Characteristics of a Good User Interface

A "Good" User Interface (UI) bridges the gap between the human user and the complex computer system. If the interface is bad, even a powerful software product will fail because people won't be able to use it.

To be considered "Good," an interface must satisfy the following **6 Key Characteristics**:

1. Speed of Learning (*Learnability*)

- **Definition:** How much time and effort does it take for a *new user* to learn how to use the system?
- **Goal:** A good UI should be intuitive. Users should not need to read a 100-page manual to perform basic tasks.
- **How to achieve:** Use familiar metaphors (e.g., a "Trash Can" icon for deleting, a "Folder" icon for storage).

2. Speed of Use (*Efficiency*)

- **Definition:** Once the user has learned the system, how quickly can they complete tasks?
- **Goal:** The interface should not slow down an *experienced user*.
- **How to achieve:**
 - Provide **Keyboard Shortcuts** (e.g., Ctrl+C, Ctrl+V).
 - Avoid excessive clicking (nested menus).
 - Support command-line options for experts.

3. Speed of Recall (*Memorability*)

- **Definition:** If a user stops using the system for a month and comes back, how much do they remember?
- **Goal:** The user should not have to re-learn the system every time.
- **How to achieve:** Use clear, meaningful labels for buttons (e.g., use "Save" instead of "Commit Transaction").

4. Error Recovery (*Forgiveness*)

- **Definition:** How easily can a user fix a mistake?
- **Goal:** A good UI should "forgive" the user for making errors.
- **How to achieve:**
 - **Undo/Redo:** Allow users to reverse actions (Ctrl+Z).
 - **Confirmation Dialogs:** Ask "Are you sure?" before deleting important data.
 - **Helpful Error Messages:** Instead of "Error 404," say "Page not found, please check the URL."

5. Consistency

- **Definition:** The interface should look and behave the same way across all screens.
- **Goal:** To build muscle memory.
- **How to achieve:**
 - **Visual Consistency:** Use the same fonts, colors, and button styles on every page.
 - **Functional Consistency:** If "Ctrl+S" saves the file in one module, it should not close the window in another module.

6. Attractiveness (*Aesthetics*)

- **Definition:** The visual appeal of the interface.
- **Goal:** To make the user comfortable and reduce eye strain.
- **How to achieve:**
 - Use a pleasing color scheme (not neon text on a black background).
 - Avoid clutter (too many buttons on one screen).

5.2: Basic Concepts of User Interface Design (UID).

Exam Status:

- **Frequency:** Low as a standalone long question, but specific concepts like "Metaphor" have appeared.
- **Relevant Question:** "What do you mean by a metaphor in UID?" (2021, 2 Marks).

Topic 5.2: Basic Concepts of UID

1. Definition of User Interface (UI)

The User Interface is the part of the computer system that the user sees, hears, and touches. It is the bridge between the human user and the computer's complex logic.

- **Input:** How the user tells the computer what to do (Keyboard, Mouse, Voice).
- **Output:** How the computer shows results to the user (Screen, Sound, Printer).

2. The Goal of UID

The main goal is to make the interaction "**Transparent.**"

- This means the user should focus on *doing their work* (e.g., writing a letter), not on *operating the computer*. The interface should feel natural and invisible.

3. Key Concept: Metaphor in UID

(Asked in 2021)

- **Definition:** A **Metaphor** in UI design means using real-world objects to represent computer concepts. By mimicking the real world, users understand how to use the software instantly without needing instructions.
- **Why use it?** It connects the "Unknown" (computer functions) to the "Known" (daily life objects).

Examples of UI Metaphors:

- **The "Desktop":** The main screen looks like a physical office desk.
- **"Trash Can" / "Recycle Bin":** Represents the function of deleting files. You "throw" files into it.

- **"Folders":** The computer uses directories, but we call them "Folders" because people know how to put papers in a physical folder.
- **"Scissors" Icon:** Represents "Cut" functionality.

4. Other Basic Concepts

- **Feedback:** The system must always tell the user what is happening.
 - *Example:* When a file is copying, show a **Progress Bar**. If the user clicks a button, the button should look "pressed."
- **Consistency:** (Covered in 5.1) Keeping the design uniform (same fonts, colors, and button locations) throughout the application.

5.3: Types of User Interfaces.

Exam Status:

- **Frequency: Diamond Tier** (Asked **5 times**: 2020, 2021, 2022, 2023, 2024).
- **Question Type:** "Classify the user interface" (10 Marks) or specific comparisons like "GUI vs Text-based" (5 Marks).
- **Key Requirement:** You must describe the three main types: **Command Language**, **Menu-Based**, and **Direct Manipulation (GUI)**.

Topic 5.3: Types of User Interfaces

The User Interface (UI) is the mechanism by which the user interacts with the machine. We classify UIs into three major categories based on how the user gives commands:

1. Command Language Interface (CLI)

(Asked specifically in 2022 & 2024)

- **Definition:** The user interacts with the system by typing specific text **commands** (keywords) at a prompt. The user must know the correct syntax and spelling.
 - **Example:** MS-DOS, Unix/Linux Terminal, Windows Command Prompt (C:\> copy file1.txt file2.txt).
 - **Advantages:**
 - **Speed for Experts:** Experienced users can type commands much faster than clicking through multiple windows.
 - **Low Resources:** Uses very little memory (RAM) and processing power.
 - **Scripting:** You can easily automate tasks by saving a list of commands in a file (script).
 - **Disadvantages:**
 - **Hard to Learn:** Users must memorize complex commands and syntax.
 - **No Forgiveness:** A single typing error (typo) causes the command to fail.
 - **High Mental Load:** The user has to remember the state of the system (e.g., "Which folder am I in right now?").
-

2. Menu-Based Interface

(Asked specifically in 2022)

- **Definition:** The user interacts by selecting an option from a predefined list (menu). The system presents all valid commands, and the user just picks one.
- **Example:** ATM machines, early mobile phones, TV remote menus.
- **Advantages:**
 - **Easy to Learn:** No need to memorize commands; just recognize the option you want.
 - **Error Reduction:** Users cannot type a "wrong" command because they can only choose valid options.
 - **Structured:** Guides the user step-by-step.
- **Disadvantages:**
 - **Slow Navigation:** For complex tasks, the user might have to go through many nested menus (Menu $\rightarrow$ Sub-menu $\rightarrow$ Sub-menu).
 - **Screen Space:** Menus take up a lot of screen space.

Types of Menus (Important for Exams):

1. **Pull-Down Menu:** A list drops down from a menu bar at the top (e.g., File, Edit).
2. **Pop-up Menu:** Appears at the cursor position (usually via Right-Click).
3. **Cascading Menu:** A secondary menu that opens to the side of the main menu item.
4. **Iconic Menu:** Using images/icons instead of text to represent choices (e.g., a Toolbar).

3. Graphical User Interface (GUI) / Direct Manipulation

(Asked in 2024)

- **Definition:** The user interacts with visual objects on the screen that mimic the real world. It relies on the WIMP model: **W**indows, **I**cons, **M**enus, and **P**ointers.
- **Example:** Windows 10/11, macOS, Android, Web Browsers.
- **Advantages:**
 - **Intuitive:** Users can "see" what they are doing (e.g., dragging a document to a folder feels like moving a physical paper).
 - **Multitasking:** Can display multiple applications (Windows) at the same time.
 - **WYSIWYG:** "What You See Is What You Get" (The document on screen looks exactly like the printed version).
- **Disadvantages:**
 - **Resource Heavy:** Requires powerful hardware (Graphics card, RAM).
 - **Development Cost:** It is much harder and more expensive to program a GUI than a CLI.

Here are the comprehensive notes for **Topic 5.3: Types of User Interfaces**.

Exam Status:

- **Frequency: Diamond Tier** (Asked **5 times**: 2020, 2021, 2022, 2023, 2024).
- **Question Type:** "Classify the user interface" (10 Marks) or specific comparisons like "GUI vs Text-based" (5 Marks).

- **Key Requirement:** You must describe the three main types: **Command Language**, **Menu-Based**, and **Direct Manipulation (GUI)**.

Topic 5.3: Types of User Interfaces

The User Interface (UI) is the mechanism by which the user interacts with the machine. We classify UIs into three major categories based on how the user gives commands:

1. Command Language Interface (CLI)

(Asked specifically in 2022 & 2024)

- **Definition:** The user interacts with the system by typing specific text **commands** (keywords) at a prompt. The user must know the correct syntax and spelling.
- **Example:** MS-DOS, Unix/Linux Terminal, Windows Command Prompt (C:\> copy file1.txt file2.txt).
- **Advantages:**
 - **Speed for Experts:** Experienced users can type commands much faster than clicking through multiple windows.
 - **Low Resources:** Uses very little memory (RAM) and processing power.
 - **Scripting:** You can easily automate tasks by saving a list of commands in a file (script).
- **Disadvantages:**
 - **Hard to Learn:** Users must memorize complex commands and syntax.
 - **No Forgiveness:** A single typing error (typo) causes the command to fail.
 - **High Mental Load:** The user has to remember the state of the system (e.g., "Which folder am I in right now?").

2. Menu-Based Interface

(Asked specifically in 2022)

- **Definition:** The user interacts by selecting an option from a predefined list (menu). The system presents all valid commands, and the user just picks one.
- **Example:** ATM machines, early mobile phones, TV remote menus.
- **Advantages:**
 - **Easy to Learn:** No need to memorize commands; just recognize the option you want.
 - **Error Reduction:** Users cannot type a "wrong" command because they can only choose valid options.
 - **Structured:** Guides the user step-by-step.
- **Disadvantages:**
 - **Slow Navigation:** For complex tasks, the user might have to go through many nested menus (Menu → Sub-menu → Sub-menu).
 - **Screen Space:** Menus take up a lot of screen space.

Types of Menus (Important for Exams):

- 1. **Pull-Down Menu:** A list drops down from a menu bar at the top (e.g., File, Edit).
- 2. **Pop-up Menu:** Appears at the cursor position (usually via Right-Click).
- 3. **Cascading Menu:** A secondary menu that opens to the side of the main menu item.
- 4. **Iconic Menu:** Using images/icons instead of text to represent choices (e.g., a Toolbar).

3. Graphical User Interface (GUI) / Direct Manipulation

(Asked in 2024)

- **Definition:** The user interacts with visual objects on the screen that mimic the real world. It relies on the **WIMP** model: **W**indows, **I**cons, **M**enus, and **P**ointers.
- **Example:** Windows 10/11, macOS, Android, Web Browsers.
- **Advantages:**
 - **Intuitive:** Users can "see" what they are doing (e.g., dragging a document to a folder feels like moving a physical paper).
 - **Multitasking:** Can display multiple applications (Windows) at the same time.
 - **WYSIWYG:** "What You See Is What You Get" (The document on screen looks exactly like the printed version).
- **Disadvantages:**
 - **Resource Heavy:** Requires powerful hardware (Graphics card, RAM).
 - **Development Cost:** It is much harder and more expensive to program a GUI than a CLI.

Comparison: GUI vs. Text-Based (CLI)

(Use this table for the 2024 "Compare characteristics" question)

Feature	Text-Based (CLI)	Graphical (GUI)
Interaction	Typing commands via keyboard.	Pointing and clicking via mouse/touch.
Ease of Learning	Hard (Must memorize syntax).	Easy (Intuitive/Visual).
Speed	Faster for experts.	Slower (Need to move hand/mouse).

Feature	Text-Based (CLI)	Graphical (GUI)
Memory Usage	Very Low.	High (Needs graphics power).
Multitasking	Difficult (Usually one task at a time).	Easy (Multiple windows).
Flexibility	High (Combinable commands).	Low (Limited to provided buttons).

5.4: Component-Based GUI Development.

Exam Status:

- **Frequency: 0 Times** in the last 5 years (2020–2024).
- **Priority: Low.** This topic has never been asked directly. It is mainly a practical concept (how you actually build a GUI using tools like Visual Basic, Java Swing, or .NET).

Topic 5.4: Component-Based GUI Development

1. Definition

Component-based GUI development is a method of building User Interfaces by assembling **pre-existing visual components** (widgets/controls) rather than drawing every pixel from scratch.

- **Analogy:** It's like building with LEGO bricks. You don't make the plastic; you just pick a "Window" brick, a "Button" brick, and a "Textbox" brick and snap them together.

2. Common GUI Components (The Building Blocks)

In any standard GUI toolkit (like Java Swing, WinForms, or Web), you use these standard components:

- Windows / Containers:**
 - The main area that holds other components (e.g., Frame, Panel, Dialog Box).
- Input Controls:**
 - **Text Box:** For typing text.
 - **Buttons:** To trigger an action (Submit, Cancel).
 - **Radio Buttons:** To select **one** option from a group (e.g., Gender: Male/Female).
 - **Checkboxes:** To select **multiple** options (e.g., Hobbies).
- Navigational Components:**
 - **Menus:** (File, Edit, View).
 - **Scrollbars:** To view content larger than the screen.
- Informational Components:**
 - **Labels:** Static text to explain what a field is.
 - **Progress Bars:** Showing task completion status.

3. Properties and Events

Each component has two main aspects:

- **Properties:** Characteristics like Color, Size, Font, and Position.
- **Events:** Actions that the component responds to (e.g., OnClick, OnMouseOver, OnKeyPress).

4. Advantages

- **Rapid Development:** Much faster than coding UI from scratch.
- **Consistency:** A standard "OK" button looks the same across the entire application.
- **Reliability:** These components are pre-tested and bug-free.

UNIT 6: Software Coding & Testing.

Topic	Frequency	Question Type	Priority
White Box vs Black Box	5 Times	"Discuss White Box/Black Box testing with example"	 Diamond
Integration Testing	4 Times	"Explain different Integration testing approaches"	 Diamond
Debugging	4 Times	"Define Debugging. Approaches & Guidelines"	 Diamond
Code Review	2 Times	"Code Inspection vs Code Walkthrough"	 Silver
Verification vs Validation	1 Time	"Define V&V"	 Silver

1. Black Box Testing (Functional Testing)

- **Definition:** Testing the system's functionality without knowing its internal structure (code).
- **Perspective:** The tester acts like a **User**. They only see inputs and outputs.

- **Analogy:** Testing a car by driving it (Does the brake work?) without opening the bonnet to check the engine.
- **Techniques Used:**
 - **Equivalence Class Partitioning:** Dividing inputs into valid and invalid groups (e.g., if valid range is 1-100, test one number inside and one outside).
 - **Boundary Value Analysis (BVA):** Bugs often hide at boundaries. If range is 1-100, we test specifically at 0, 1, 100, and 101.

2. White Box Testing (Structural / Glass Box Testing)

- **Definition:** Testing the internal logic, code structure, and paths of the system.
- **Perspective:** The tester acts like a **Developer**. They look at the source code.
- **Analogy:** A mechanic opening the car bonnet to check if the wires are connected correctly.
- **Techniques Used:**
 - **Statement Coverage:** Ensure every line of code is executed at least once.
 - **Branch/Decision Coverage:** Ensure every `True` and `False` branch of an `IF` statement is taken.
 - **Path Coverage:** Ensure every possible path through the code is tested.
 - **Cyclomatic Complexity:** A metric to calculate the number of independent paths.

6.8: Difference between Black Box and White Box

Feature	Black Box Testing	White Box Testing
Focus	Functionality (External behavior).	Logic/Structure (Internal code).
Knowledge Needed	No knowledge of code required.	Knowledge of programming is mandatory.
Done By	Testers / Users.	Developers.
Main Goal	To check if requirements are met.	To check logic, loops, and branches.
Techniques	Boundary Value, Equivalence Class.	Statement Coverage, Path Coverage.
Cost	Low skilled testers (Cheaper).	Highly skilled developers (Expensive).

Definition: Once individual modules are tested (Unit Testing), we combine them to build the complete system. **Integration Testing** checks the *interfaces* and *data flow* between these combined modules.

Approaches (Methodologies):

1. **Big Bang Integration:**

- Wait until all modules are built, then join them all at once and test.
- *Disadvantage:* If it crashes, it's impossible to find the cause because everything is mixed up.

2. **Top-Down Integration:**

- Start with the Main Module (Top) and add lower modules one by one.
- *Stubs:* Since lower modules might not be ready, we use dummy code called "**Stubs**" to simulate them.

3. **Bottom-Up Integration:**

- Start with the lowest utility modules (Bottom) and move up.
- *Drivers:* Since the top main module isn't ready to call the bottom ones, we use dummy code called "**Drivers**" to call them.
- *Advantage:* Easier to find bugs in specific features.

4. **Mixed / Sandwich Integration:**

- Uses both Top-Down and Bottom-Up simultaneously to save time.

Topic 6.9 & 6.10: Debugging

*Exam Status: **Diamond Tier** (Asked in 2020, 2021, 2023, 2024).*

1. Definition: Debugging is the process of **locating** and **fixing** errors (bugs) after testing has detected them.

- *Note:* Testing *finds* the bug; Debugging *fixes* it.

2. **Debugging Approaches (Strategies):** *(Asked in 2020 & 2024)*

1. **Brute Force Method:**

- The most common but least efficient. The developer puts `print()` statements everywhere or dumps memory to see what's happening.

2. **Backtracking:**

- Start at the line where the error occurred and trace the code **backward** logic-wise until the cause is found. Good for small programs.

3. **Cause Elimination (Induction/Deduction):**

- Develop a hypothesis ("I think variable X is null") and devise a test to prove or disprove it.

3. **Debugging Guidelines:**

- Fix the **cause**, not just the **symptom**.
- One bug fix often introduces a new bug (Regression), so always re-test.
- Don't just patch the code; understand *why* it failed.

Topic 6.2: Code Review (Inspection vs. Walkthrough)

*Exam Status: **Silver Tier** (Asked in 2021, 2024).*

Feature	Code Walkthrough	Code Inspection
Formality	Informal / Casual.	Very Formal / Strict.
Process	The author reads the code to the team, and they ask questions.	A trained moderator leads the meeting. The author is just a participant.
Goal	Learning and understanding.	Finding defects and bugs.
Preparation	Team doesn't prepare much beforehand.	Team studies the code <i>before</i> the meeting.

UNIT 7: Software Reliability & Quality.

Topic	Frequency	Question Type	Priority
Define Software Reliability	5 Times	"Define software reliability" (2 Marks)	 Diamond
Hardware vs. Software Reliability	2 Times	"Differentiate between Hardware and Software reliability"	 Gold
Software Quality	2 Times	"What is software quality?", "Quality Management System"	 Gold
Reliability Metrics	1 Time	Mentions of MTBF/MTTR in short notes	 Silver

Topic 7.1: Software Reliability

Exam Status: **Diamond Tier** (Asked every year: 2020, 2021, 2022, 2023, 2024).

1. Definition

Software Reliability is defined as the **probability** that a software system will perform its required functions without failure for a specified period of time in a specified environment.

- **Simpler terms:** Can I trust this software not to crash right now?
- **Difference from Quality:** Quality is broad (speed, looks, features). Reliability is specific (does it crash?).

2. Hardware vs. Software Reliability (Crucial 5-Mark Question)

(Asked in 2021 & 2022) You must be able to explain why software fails differently from hardware.

Feature	Hardware Reliability	Software Reliability
Failure Cause	Fails due to wear and tear (rust, friction, age).	Fails due to design errors (bugs) or bad logic.
Wear Out	Hardware wears out over time.	Software does not wear out . It works exactly the same the 1st time and the 1000th time.
Bathtub Curve	Follows the "Bathtub Curve" (High failure at start, low in middle, high at end).	Failure rate drops as bugs are fixed, but can spike again if a bad update is released.
Repair	Broken parts are replaced with spare parts.	Bugs are fixed by changing the design/code.
Time	Reliability decreases with time (aging).	Reliability typically increases with time (as bugs are found and fixed).

Topic 7.2: Reliability Metrics

To measure "how reliable" software is, we use specific metrics.

- Mean Time Between Failures (MTBF):**
 - The average time the system runs before it crashes.
 - Formula:* $MTBF = MTTF + MTTR$
 - High MTBF is Good.*
- Mean Time To Failure (MTTF):**
 - The average time the system works correctly.
- Mean Time To Repair (MTTR):**
 - The average time it takes to fix the system once it has crashed.
 - Low MTTR is Good.*
- Rate of Occurrence of Failure (ROCOF):**
 - How frequently do failures happen? (e.g., 2 failures per 100 hours).

Topic 7.4 & 7.5: Software Quality & Quality Management

(Asked in 2020 & 2024)

1. What is Software Quality?

Software Quality is the degree to which a system meets the specified requirements (SRS) and customer expectations.

- **McCall's Quality Factors:** To measure quality, we look at:
 - **Correctness:** Does it do what it's supposed to?
 - **Reliability:** Does it crash?
 - **Efficiency:** Does it waste RAM/CPU?
 - **Usability:** Is it easy to use?
 - **Maintainability:** Is it easy to fix?

2. Software Quality Management System (QMS)

(Asked as a 10-mark question in 2020) A QMS is the system of processes used to ensure quality. The two most famous standards are **ISO 9000** and **SEI CMM**.

A. ISO 9000 Series (ISO 9001):

- A general international standard for quality management (not just for software).
- **Focus:** It certifies the **process**, not the product. If you follow a good process (documenting, testing, reviewing), you are assumed to produce a good product.
- **Certificate:** A company gets "ISO 9001 Certified" after an audit.

B. SEI CMM (Capability Maturity Model):

- Developed specifically for **Software**. It rates a company's software process maturity on a scale of 1 to 5.
 - **Level 1 (Initial):** Chaotic. No plans. Heroic effort by individuals.
 - **Level 2 (Repeatable):** Basic project management is in place.
 - **Level 3 (Defined):** Processes are documented and standardized across the company.
 - **Level 4 (Managed):** Processes are measured quantitatively (using metrics like LOC/FP).
 - **Level 5 (Optimizing):** Continuous improvement based on data.

Topic 7.6: Error Seeding (Brief)

(Asked in 2020 & 2023 as a definition)

- **Definition: Error Seeding** is a technique used to estimate the number of hidden errors in a system.
- **How it works:**
 1. The manager intentionally inserts (seeds) 50 known "fake bugs" into the code.
 2. The testers test the software and find bugs.
 3. If testers find only 10 of the fake bugs (20%), we assume they likely found only 20% of the *real* bugs too.
 4. It helps guess how many real bugs are left remaining.

Final Exam Strategy for You

You have now completed the entire syllabus. Here is your **Last-Minute Cheat Sheet** for the exam.

The "Must-Answer" Questions (High Marks):

1. **Unit 1: Spiral Model** ("Meta-Model") and **Prototyping Model**.
2. **Unit 2: COCOMO** (Modes & Formula) and **Risk Management**.
3. **Unit 3: SRS** (Characteristics of Good SRS).
4. **Unit 4: DFD** (Symbols/Levels) and **Cohesion/Coupling** (Types).
5. **Unit 5: Types of UI** (Command vs GUI).
6. **Unit 6: White Box vs Black Box Testing**.